OOP Project 25/26

LEEC/MEEC

UNO Game Engine with Scripted Execution

Contents

Overview

In this project you will design and implement a modular UNO game engine in Java. The project has two phases:

- **Phase 1**: Implement a UNO game engine that:
  - reads a fixed deck of cards from a text file (the exact order of cards is given),
  - reads a scripted sequence of player moves from a text file,
  - simulates a game according to the specification below,
  - writes a detailed, machine-readable event log to standard output.
- **Phase 2**: Extend your engine following the *Open–Closed Principle* (open for extension, closed for modification).

1 Phase 1: UNO game

UNO is a turn-based card game in which each player tries to be the first to empty their hand. On each turn, a player may play a compatible card or draw from the draw pile. Cards may match by color or by rank, and some special cards modify the flow of the game by skipping turns, reversing the direction of play, forcing draws, or allowing a new color to be chosen.

In this project, the game is simulated through a scripted execution based on input files: one file defines the deck, and another defines the sequence of player commands.

1.1 Game rules

This section specifies the rules of the UNO-like card game you must implement.

1.1.1 Cards

Each card has:

- a **color**, one of:
  - Red (denoted `R`),
  - Yellow (denoted `Y`),
  - Green (denoted `G`),
  - Blue (denoted `B`),
  - Wild (denoted `W`) — used only for wild cards.
- a **rank** (or type), one of:
  - number ranks: `0`, `1`, . . . , `9`,
  - action ranks: `SKIP`, `REVERSE`, `DRAW_TWO`,
  - wild ranks: `WILD`, `WILD_DRAW_FOUR`.

A card is written as a text code `<color>-<rank>`, for example:

- `R-5`, `Y-0`, `G-SKIP`, `B-REVERSE`,
- `W-WILD`, `W-WILD_DRAW_FOUR`.

1.1.2 Game Setup

A single game uses:

- a fixed number of players (between 2 and 6), each identified by an integer id: `0`, `1`, . . . ,
- a **draw pile**: an ordered pile of cards from which players draw,
- a **discard pile**: an ordered pile onto which played cards are placed,
- a current direction of play:

– clockwise (forwards) or counter-clockwise (backwards).

At the start of the game:

1. A *deck file* is read.
2. The first card from the deck file becomes the initial top card of the discard pile (face up). We will call this the *initial discard*.
3. The remaining cards from the deck file form the initial draw pile, preserving the order in which they appear in the file.
4. Each player is dealt a fixed number of cards from the draw pile, one card at a time, in player order starting from player `0`. Unless otherwise stated, use `7` cards per player.
5. Player `0` plays first.
6. The initial direction of play is clockwise.
7. The current color is initially set to the color of the initial discard. If the initial discard is a wild card, the program must abort with an error message. If the initial discard is an action card such as `SKIP`, `REVERSE`, or `DRAW_TWO`, its effect is not applied at setup; only its color and rank are used as the initial top discard.

Each player's hand is ordered according to the order in which cards are received. During the initial deal, cards are distributed one at a time in player order, starting from player `0`. In other words, if there are four players, the first round gives one card to players `0`, `1`, `2`, and `3`, in that order; the second round again gives one card to players `0`, `1`, `2`, and `3`; and so on, until each player has received the required number of cards. Each dealt card is appended to the end of that player's hand. Likewise, whenever a player draws a card during the game, that card is appended to the end of the hand.

This ordering is important to ensure that the output remains consistent with the provided input files and reproducible across different implementations.

1.1.3 Turn Structure

On a normal turn, the current player must do exactly one of the following:

- **Play** one card from their hand, provided that the card is legally playable.
- **Draw** one card from the draw pile.

If the player plays a wild card that requires a color choice, then the player must immediately perform the additional action:

- **Choose color** by selecting the new active color.

1.1.4 Playability of Cards

A card is playable if at least one of the following holds:

- its color matches the current color,
- its rank matches the rank of the top card of the discard pile,
- it is a wild card.

For wild cards, the current color is determined by the most recent color choice after a wild card was played.

1.1.5 Effects of Cards

The card effects required in Phase 1 are the following:

Number cards (`0`–`9`)
: These cards have no special effect beyond being placed on the discard pile. After such a card is played, the turn advances normally to the next player.

`SKIP`
: The next player loses their turn.

`REVERSE`
: The direction of play is reversed. In a two-player game, this behaves like a skip: the same player plays again.

`DRAW_TWO`
: The next player draws 2 cards and loses their turn.

`WILD`
: The player chooses the new current color.

`WILD_DRAW_FOUR`
: The player chooses the new current color. The next player draws 4 cards and loses their turn.

1.1.6 Winning Condition

A player wins immediately when their hand becomes empty after a valid play. At that point, the game ends.

1.1.7 Draw Pile Exhaustion

If a player must draw a card and the draw pile is empty, the game ends immediately without a winner. This situation may occur whenever the deck file does not contain enough cards to support the full execution of the game, including the initial discard, the initial hands dealt to the players, and all subsequent draw actions required by the script.

1.2 Input Files

Your program must read two text files: a deck file and a script file.

1.2.1 Deck File

The deck file lists all cards in order, one per line, and:

- blank lines may appear and should be ignored,
- lines beginning with `#` are comments and should be ignored,
- inline comments beginning with `#` may also be supported.

Example:

```
# first card = initial discard
R-5
G-2
Y-SKIP
W-WILD
B-9
...
```

In this example `R-5` is the initial discard, and the remaining cards form the initial draw pile in the given order.

1.2.2 Script File

The script file contains the sequence of player commands to execute. Allowed commands are:

```
PLAYER <id> PLAY <index>
PLAYER <id> DRAW
PLAYER <id> COLOR <R|Y|G|B>
```

where:

- `<id>` is the player id,
- `<index>` is the position of the card in that player's current hand,
- `COLOR` is only valid immediately after a wild card that requires color choice has been played.

Example:

```
PLAYER 0 DRAW
PLAYER 1 PLAY 0
PLAYER 1 COLOR R
..
```

As with the deck file:

- blank lines may be ignored,
- comment lines starting with `#` should be ignored,
- inline comments are **not** supported.

Recall that:

- in a `PLAY` command, the given `<index>` refers to the current position of the card in that player's hand, using zero-based indexing.
- cards are dealt in rounds, one card per player at a time, starting from player `t`he ordering of the cards must be respected (c.f. Section 1.1.2) so that the output remains consistent with the provided input files and reproducible across different implementations.

1.3 Command Semantics and Validation

For each script command, your engine must verify that the command is legal in the current state.

1.3.1 Common Conditions

In general, a command is invalid if:

- it refers to a player who does not exist,
- it is not that player's turn,
- the command is not allowed in the current game phase,
- an index is outside the valid range for that player's current hand.

1.3.2 `PLAY`

A `PLAY` command is valid only if:

- the specified card index exists in the current player's hand,
- the selected card is playable according to the current color and top discard,
- the game is in a phase where playing a card is allowed.

1.3.3 DRAW

A DRAW command is valid only if:

- it is the current player's turn,
- the game is in a phase where drawing is allowed.

1.3.4 COLOR

A COLOR command is valid only if:

- it is the correct player's turn to choose a color,
- the game is in a phase waiting for a color choice,
- the chosen color is one of R, Y, G, B.

If a command violates these conditions, the engine must not change the game state and must report an error event in its output (c.f. Section **??**).

1.4 Output: Event Log

Your program must write a detailed event log to standard output, in plain text. This log will be used to verify that the engine follows the rules. You should not include additional unrelated human-readable lines. The following structure and event categories must be present.

1.4.1 Startup Header

Before the machine-readable event log begins, the program must print a short startup header indicating the parameters used to run the program. The format must be:

```
Running <mainClass> with:
   Deck file:           <deckFile>
   Script file:         <scriptFile>
   Nb players:          <playerCount>
   Nb cards per player: <cardsPerPlayer>
```

where:

- <mainClass> is the fully qualified name of the Java class containing the main method, that is, the complete class name including its package(s) name;
- <deckFile> is the deck file provided in the command line;
- <scriptFile> is the script file provided in the command line;
- <playerCount> is the number of players;
- <cardsPerPlayer> is the number of cards dealt to each player at the beginning of the game.

For example, if the program is run through class uno.Main, then <mainClass> should be printed as uno.Main.

1.4.2 Game Start

At the beginning of the run, before processing any script commands, print:

```
GAME_START players=<n>
EVENT TOP_CARD <card> color=<color>
EVENT HAND player=<id> cards=<card1> <card2> ... <cardk>
...
EVENT TURN_START player=<id>
```

where:

- <n> is the total number of players,
- <card> is the current top card of the discard pile,

- `<color>` is the current color,
- each `EVENT HAND` line shows the current hand of each player, with cards written as `<color>-<rank>` separated by spaces,
- the final line identifies which player will act first.

The `EVENT HAND` lines must be printed only once, at the beginning of the execution, as part of the initial game state. They are not printed again after subsequent commands, since the order of cards in each hand is fixed (cf. Section 1.1.2). This helps ensure that the game evolution can be reproduced unambiguously from the input files, since the game is fully deterministic given the deck file, script file, player count, and cards per player.

1.4.3 Command Logging

For each command read from the script file (whether valid or not), print:

```
EVENT COMMAND line="<original script line>"
```

Use the original command text as read from the script.

1.4.4 Action Events

For every valid command, the engine must print one or more action events describing the effect of that command on the game state. In Phase 1, the following action-event categories are required:

```
EVENT PLAY_CARD <description>
EVENT DRAW_CARD <description>
EVENT CHOOSE_COLOR <description>
EVENT TURN_ADVANCE <description>
```

The output format must follow a stable and consistent wording so that outputs can be compared automatically.

General rules. The following rules define the expected event structure for the most common Phase 1 situations:

- after a successful `PLAY` command, an `EVENT PLAY_CARD` line must be printed,
- after a successful `DRAW` command, an `EVENT DRAW_CARD` line must be printed,
- after a successful `COLOR` command, an `EVENT CHOOSE_COLOR` line must be printed,
- whenever the current player changes, an `EVENT TURN_ADVANCE` line must be printed.

In a game with 4 players, assuming that the current direction is clockwise, the output must follow the patterns:

Case 1: normal play. For a normal card with no additional effect, the output must follow the pattern:

```
EVENT COMMAND line="PLAYER 0 PLAY 2"
EVENT PLAY_CARD Player 0 played R-5
EVENT TURN_ADVANCE Next player: 1
```

Case 2: normal draw. When a player explicitly draws from the draw pile, the output must follow the pattern:

```
EVENT COMMAND line="PLAYER 0 DRAW"
EVENT DRAW_CARD Player 0 draws 1 card (Y-2)
EVENT TURN_ADVANCE Next player: 1
```

Case 3: `SKIP`**.** When a player plays `SKIP`, the next player loses their turn. The output must follow the pattern:

```
EVENT COMMAND line="PLAYER 1 PLAY 2"
EVENT PLAY_CARD Player 1 played SKIP
EVENT TURN_ADVANCE Next player: 3
```

In a two-player game, this means that the same player plays again.

Case 4: `REVERSE`**.** When a player plays `REVERSE`, the direction changes. The output must follow the pattern:

```
EVENT COMMAND line="PLAYER 0 PLAY 1"
EVENT PLAY_CARD Player 0 played REVERSE
EVENT TURN_ADVANCE Next player: 3
```

In a two-player game, `REVERSE` behaves like `SKIP`, so the same player plays again.

Case 5: `DRAW_TWO`**.** When a player plays `DRAW_TWO`, the next player draws 2 cards and loses their turn. This forced draw must be reflected in the `EVENT PLAY_CARD` line. A separate `EVENT DRAW_CARD` line is not required for those 2 cards. The output must follow the pattern:

```
EVENT COMMAND line="PLAYER 0 PLAY 4"
EVENT PLAY_CARD Player 0 played DRAW_TWO; Player 1 draws 2 and is skipped
EVENT TURN_ADVANCE Next player: 2
```

In a two-player game, this means that the player who played `DRAW_TWO` will play again.

Case 6: `WILD`**.** For a wild card, the play event and the color-choice event are separate. The output must follow the pattern:

```
EVENT COMMAND line="PLAYER 1 PLAY 0"
EVENT PLAY_CARD Player 1 played WILD (color will be chosen)
EVENT COMMAND line="PLAYER 1 COLOR R"
EVENT CHOOSE_COLOR Player 1 chose color RED
EVENT TURN_ADVANCE Next player: 0
```

Case 7: `WILD_DRAW_FOUR`**.** When a player plays `WILD_DRAW_FOUR`, the next player draws 4 cards and loses their turn, and the current player must choose the new color. As with `DRAW_TWO`, the forced draw must be reflected in the `EVENT PLAY_CARD` line. A separate `EVENT DRAW_CARD` line is not required for those 4 cards. The output must follow the pattern:

```
EVENT COMMAND line="PLAYER 0 PLAY 2"
EVENT PLAY_CARD Player 0 played WILD_DRAW_FOUR; Player 1 draws 4 and is skipped
EVENT COMMAND line="PLAYER 0 COLOR Y"
EVENT CHOOSE_COLOR Player 0 chose color YELLOW
EVENT TURN_ADVANCE Next player: 2
```

In a two-player game, this means that the player who played `WILD_DRAW_FOUR` will play again.

Forced draws caused by `DRAW_TWO` or `WILD_DRAW_FOUR` are described in the corresponding `EVENT PLAY_CARD` line. No separate `EVENT DRAW_CARD` line should be printed for those forced draws.

1.4.5 Error Events

If a command is invalid, the program must print an error event in the following format:

```
EVENT ERROR <description>
```

For the standard Phase 1 situations, the wording of the error descriptions must follow the forms below. Here, `<id>` is the player identifier, `<card>` is the respective card, and `<index>` is the card position selected by the player.

```
EVENT ERROR Not player <id> turn
EVENT ERROR Card <card> is not playable
EVENT ERROR Invalid card index <index>
EVENT ERROR Cannot choose color - no wild card was played
EVENT ERROR Must choose a color before playing another card
EVENT ERROR Must choose a color before drawing
EVENT ERROR Only player <id> can choose the color
EVENT ERROR Cannot choose WILD as a color
```

For example, the same error formats with concrete values would be:

```
EVENT ERROR Not player 2 turn
EVENT ERROR Card Y-2 is not playable
EVENT ERROR Invalid card index 8
EVENT ERROR Only player 1 can choose the color
```

Other error situations may arise; in such cases, the emitted error message should still be clear, consistent, and informative.

After the first invalid command, the program must terminate immediately.

1.4.6 Game End

When a player wins (their hand becomes empty), print:

```
EVENT GAME_END Player <id> wins
EVENT WINNER player=<id>
GAME_END
```

If the script ends without any player having won, you must still print:

```
GAME_END
```

and you may optionally print an additional `EVENT ERROR` explaining that no winner was determined.

If the game ends without a winner because, for instance, no card is available to draw (recall Section 1.1.7), the program must print:

```
EVENT GAME_END No cards available to draw
GAME_END
```

1.5 Requirements Summary

In Phase 1 you must:

- implement a UNO game engine that follows the rules specified in the previous sections,
- read a deck file to determine the exact order of cards (support code given in Section 2),
- read a script file to determine the exact sequence of moves (support code given in Section 2),
- maintain an internal game state (players, hands, draw pile, discard pile, current player, direction, current color, winner),
- enforce the rules (including legality of moves and action card effects),
- write a structured event log to standard output as specified in Section 1.4.

1.6 Running your project from the command line

In Phase 1, the program must be invoked from the command line with the following format:

```
java -jar project-v1.jar <deckFile> <scriptFile> <playerCount> [<cardsPerPlayer>]
```

where:

- `<deckFile>` is the path to the deck file,
- `<scriptFile>` is the path to the script file,

- `<playerCount>` is the number of players,
- `<cardsPerPlayer>` is optional and, if omitted, the default value is 7.

Thus, a typical execution may look like:

```
java -jar project-v1.jar deck.txt script.txt 2
```

or, explicitly indicating the number of cards per player:

```
java -jar project-v1.jar deck.txt script.txt 2 7
```

The deck and script files may be given using relative or absolute paths. For example, if the files are stored in a folder called `TESTS` placed next to the executable, then the following invocations are valid:

```
java -jar project-v1.jar ./TESTS/deck.txt ./TESTS/script.txt 2
```

or simply:

```
java -jar project-v1.jar TESTS/deck.txt TESTS/script.txt 2
```

Do not hardcode file or directory locations in your solution. The program must work independently of the machine on which it is executed.

1.6.1 Phase 1 Execution Script

In addition to the command-line interface above, each group implementing Phase 1 must submit a small executable script file called:

`phase1-script.sh`

This file must contain the 5 examples for Phase 1 demonstrations with input and output files provided by teachers (on Fenix) as test uses.

The file `phase1-script.sh` should be a valid shell script and should begin with an appropriate shebang line, such as:

```
#!/bin/sh
```

A typical structure is therefore:

```
#!/bin/sh

# create the output folder if needed
mkdir -p output

echo "=================================================="
echo "Example 1"
echo "=================================================="
java -jar project-v1.jar input/ex1-deck.txt input/ex1-script.txt 2 > output/out1.txt

echo "=================================================="
echo "Example 2"
echo "=================================================="
java -jar project-v1.jar input/ex2-deck.txt input/ex2-script.txt 2 > output/out2.txt

...
```

The command-line parameters of the program depend on the specific example and must therefore be adjusted accordingly by the students. The script should execute the 5 examples provided in the `input` folder and produce the corresponding results in the `output` folder (c.f. Section 4.1 - **Required ZIP structure**).

2 Provided Support Code for File Reading

To keep the focus of the project on object-oriented design and on the implementation of the UNO engine itself, the instructors provide small auxiliary classes for reading the input files. In particular, the following support classes are included:

- a `DeckLoader` class, responsible for reading the deck file;
- a `ScriptParser` class, responsible for reading the script file line by line.

These classes are intended only as support code for file input. They do not implement the game logic itself; instead, they simply reprint to the terminal the information read from the input files.

Students are free to use and adapt them, replacing those `print` instructions by the initialization of the internal objects required by their own solution.

The files are documented, and the comments in the source code explain how they can be compiled and executed. They are provided in a separate `.zip` file together with this project description.

3 Phase 2: Extensions and Open–Closed Principle

In Phase 2 you will extend your UNO engine beyond the classic Phase 1 behavior. The goal is not only to add functionality, but also to demonstrate that your design supports extension without requiring rewrites of the original implementation.

In particular, your Phase 2 design may make it possible to add new cards, new rule variants, new execution modes, or additional runtime components by *adding new classes and code*, rather than by rewriting Phase 1 engine. This is an application of the *Open–Closed Principle*: the system should be open for extension and closed for modification.

Phase 2 is an **optional bonus component** worth up to **1 additional point/value** on your final mark. It is not required in order to obtain the maximum regular project mark of **8 points**, which can already be achieved through a complete and strong Phase 1 submission.

To complete Phase 2, you must implement either **one** extension, worth up to **0.5 points**, or **two** extensions, each worth up to **0.5 points**. For bonus evaluation purposes, at most two extensions will be considered. **If two extensions are implemented, they should come from different categories and should rely on different object-oriented design ideas or design patterns**. The five available categories are described in detail in the following Sections 3.1–3.5.

3.1 New Card Effects

You can implement one of the following three card effects:

1) `DRAW_THREE`
When this card is played, the next player draws 3 cards and loses their turn.

2) `STEAL_CARD`
When this card is played, the current player steals a random card from another player who has at least two cards. The target player is selected automatically and randomly between valid targets.

3) `SWAP_HANDS`
When this card is played, the current player swaps hands with another player. The target player is selected automatically and randomly between valid targets.

The card must:

- have a clear textual representation in the deck file,
- carry its own behavior through a dedicated effect implementation,
- integrate with the rest of the engine in a sensible way.

3.2 Rule Variants

You can implement one of the following two rule variants:

1) Crazy Ruleset
A more aggressive mode in which:

- `SKIP` skips the next two players in games with at least four players,
- `DRAW_TWO` forces the next player to draw 3 cards and lose their turn,
- the rest of the behavior remains compatible with classic UNO.

2) Speed Ruleset
A faster mode in which:

- `DRAW_TWO` is weakened so that the next player draws only 1 card and loses their turn,
- the rest of the behavior remains compatible with classic UNO.

3.3 State-Based Flow

You can implement one of the following two state-based flow extentions:

1) 7–0 Variant
A variant inspired by common UNO house rules in which:

- when a player plays a `7`, they must swap hands with a randomly chosen other player,
- when a player plays a `0`, all players pass their hands in the current direction of play,
- the rest of the behavior remains compatible with classic UNO.

2) Jump-In Variant
A variant in which:

- a player may play out of turn if they hold a card that matches the current top discard exactly,
- when such a jump-in occurs, that player immediately becomes the active player,
- the rest of the behavior remains compatible with classic UNO.

Note that this variant modifies the normal Phase 1 turn-validation rule, since a move that would normally be rejected as out-of-turn may become valid under the jump-in condition.

3.4 Auxiliary Runtime Components

You may extend the system with one additional runtime service:

1) Game Statistics
A component that collects information about the execution of the game, such as the number of cards played by each player, the number of cards drawn by each player, the frequency of the different card types played, the identity of the winner, and the duration of the game.

2) Game Recorder
A component that records the sequence of relevant game events, such as card plays, explicit draw actions, color choices, turn changes, and the end of the game, and stores that information for later inspection or replay.

3.5 Alternative Execution Modes

A valid solution in this category must define two explicit AI decision strategies and use them in one of the execution modes described below. The AI strategies to provide are:

- a **random strategy**, which chooses randomly among the legal moves;
- an **aggressive strategy**, which always plays if possible and prefers action cards such as `SKIP`, `REVERSE`, or `DRAW_TWO`.

The execution modes are:

1) Hybrid scripted/AI mode
A mixed execution mode in which the deck file is still used as input, the script file controls only some players, and the remaining players are controlled automatically according to the specified AI strategies.

In this mode, the command-line parameters must explicitly indicate which players are AI-controlled and which strategy is assigned to each of them. Any player not identified as AI-controlled is assumed to be controlled through the script file.

2) Autoplay mode
A fully automatic execution mode in which the game runs from start to finish without manual intervention, by assigning AI strategies to all players.

In this mode, the deck file remains part of the input and must still be used to define the initial discard and the draw pile. By contrast, the script file is not used, since the sequence of actions is generated automatically by the program according to the selected AI strategies.

The command-line parameters must explicitly indicate which strategy is assigned to each player.

3.6 Requirements Summary

In Phase 2 you must:

- implement at least one extension beyond classic Phase 1 behavior,
- if you implement two extensions, they should come from different categories and illustrate different object-oriented design strategies,
- ensure that classic UNO (the Phase 1 behavior) continues to work unchanged,
- structure your solution so that Phase 1 and Phase 2 are delivered as separate executable JAR files. The Phase 2 solution should build on the Phase 1 solution by adding new types and packages, following the Open–Closed Principle,
- provide a short report (1–2 pages per extension) explaining how each extension was implemented, how it integrates with the original design, and what each listed execution demonstrates (c.f. Section 3.7.1).

You are free to provide one `main` program per extension, or a single configurable entry point, as long as the behavior is documented clearly.

3.7 Running Phase 2 Extensions from the Command Line

Phase 2 is delivered through a separate executable:

```
java -cp project-v2.jar:project-v1.jar <Main-Class> ...
```

The existence of a separate `project-v2.jar` reflects the fact that Phase 2 extends the Phase 1 solution while preserving the Phase 1 executable independently. In other words, `project-v1.jar` must still run the classic Phase 1 solution, whereas `project-v2.jar` is used to demonstrate the additional Phase 2 extensions.

Since Phase 2 allows different kinds of extensions, groups are free to organize their internal code as they wish. However, for evaluation purposes, every implemented Phase 2 extension must be runnable through a **standard command-line interface**, so that the instructors can inspect and test it without having to read or modify the submitted code beforehand.

Since Phase 2 allows different kinds of extensions, groups are free to organize their internal code as they wish. However, for evaluation purposes, every implemented Phase 2 extension must be runnable through a **standard command-line interface**, so that the instructors can inspect and test it without having to read or modify the submitted code beforehand.

The required format is:

```
java -cp project-v2.jar:project-v1.jar <Main-Class> \
     -p2 <extensionType> <extensionName> <deckFile> [<scriptFile>] \
     <playerCount> [<extraParameters>] [<cardsPerPlayer>]
```

where:

- `<Main-Class>` is the fully qualified class name of the class containing the `main` method used to execute the extension;
- `<extensionType>` identifies the category of extension being demonstrated;
- `<extensionName>` identifies the specific extension;
- `<deckFile>` is the path to the deck file used in the execution;
- `<scriptFile>` is the path to the script file used in the execution, whenever the selected execution mode requires one;
- `<playerCount>` is the number of players;
- `<extraParameters>` represents any additional parameters required by the specific extension or execution mode, such as the assignment of AI strategies to players – a convenient convention for these parameters is to use a comma-separated list of assignments of the form `<playerId>:<strategy>` (see examples below);
- `<cardsPerPlayer>` is optional and, if omitted, the default value is 7.

As in Phase 1, the deck and script files may be given using relative or absolute paths. The program must not rely on hardcoded locations.

Special cases: hybrid and automatic execution modes. For execution modes involving AI-controlled players, the deck file always remains part of the input, since it determines the initial discard and the draw pile. In a **hybrid scripted/AI mode**, the script file is still used, but it should contain only the commands for the players that are controlled by the script. The actions of AI-controlled players are generated automatically by the program according to the selected AI strategy. In a **fully automatic autoplay mode**, the script file is not used, since all player actions are generated automatically by the program. In that case, the command-line invocation should omit the script file and follow a dedicated format for autoplay execution.

Allowed extension categories and names. The following extension categories and extension names must be used:

`<extensionType>`	`<extensionName>`	**See**
`card`	`DRAW_THREE`	Section 3.1
	`STEAL_CARD`	
	`SWAP_HANDS`	
`ruleset`	`CrazyRuleset`	Section 3.2
	`SpeedRuleset`	
`state`	`SevenZeroVariant`	Section 3.3
	`JumpInVariant`	
`runtime`	`GameStatistics`	Section 3.4
	`GameRecorder`	
`mode`	`HybridAI`	Section 3.5
	`Autoplay`	

Assuming that the fully qualified class name of the class containing the `main` method is `uno.v2.Main` (this is only an example and not a requirement), typical executions may look as follows:

```
java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 card DRAW_THREE deck-draw3.txt script-draw3.txt 2

java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 ruleset CrazyRuleset deck-crazy.txt script-crazy.txt 3

java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 state SevenZeroVariant deck-sevenzero.txt script-sevenzero.txt 4

java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 runtime GameStatistics deck-stats.txt script-stats.txt 2

java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 mode HybridAI deck-hybrid.txt script-hybrid.txt 4 1:random,3:aggressive

java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 mode Autoplay deck-ai.txt 4 0:random,1:aggressive,2:random,3:aggressive
```

For AI-based execution modes, a convenient convention for `<extraParameters>` is to use a comma-separated list of assignments of the form `<playerId>:<strategy>`. For example, `1:random,3:aggressive` means that players 1 and 3 are AI-controlled, using the random and aggressive strategies respectively.

Groups may internally provide one `main` class or several different `main` classes. This is an internal design decision and is left to the students.

However, for evaluation purposes, each implemented Phase 2 extension must be runnable **independently**, using the same JAR file but different command-line parameters, as shown in the examples above. In particular, if a group implements two different extensions, the instructors must be able to execute and inspect each one separately, without having to modify the submitted code or manually determine which `main` class should be used.

Therefore, independently of whether the submitted solution uses a single configurable entry point or several different `main` classes, the group must provide one explicit command-line invocation for **each** extension they want to present for evaluation.

3.7.1 Phase 2 Execution Script

In addition to the command-line interface above, each group implementing Phase 2 must submit a small executable script file called:

`phase2-script.sh`

This file must contain all the Phase 2 demonstrations that the group wishes to be considered during evaluation.

General purpose. The goal of this script is to allow the instructors to run all the submitted Phase 2 demonstrations directly, without having to inspect or edit the code beforehand. The script should therefore be self-contained, easy to read, and trivial to execute.

Required content. The script must contain complete command-line invocations for the implemented Phase 2 extensions. Additionally, groups must provide:

- **5 different execution examples** for each implemented Phase 2 extension;
- therefore, **5 executions** in `phase2-script.sh` if exactly one extension is implemented;
- and **10 executions** in `phase2-script.sh` if two extensions are implemented.

Each execution must correspond to one concrete demonstration scenario and must be directly runnable.

Associated input files. For each listed execution, the group must also provide the corresponding input files, namely:

- the deck file,
- the script file, whenever that execution mode requires one,
- and any other files or parameters required by the command line.

The provided examples should be **meaningful and varied**. In particular, they should not all be trivial variations of the same test. Instead, they should illustrate different relevant aspects of the implemented extension.

Required structure of the script. The script must be organized so that each execution is clearly identifiable by the instructors. In particular, before each command, the script must print:

- a clear header identifying the extension being demonstrated;
- the number of the example being executed;
- and a visible separator between executions.

As for Phase 1, the file `phase2-script.sh` should be a valid shell script, so a typical structure is like:

```
#!/bin/sh

# create the output folder if needed
mkdir -p output

echo "================================================"
echo "Extension: card DRAW_THREE -- Example 1"
echo "================================================"
java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 card DRAW_THREE input/deck1.txt input/script1.txt 2 > output/out1.txt

echo "================================================"
echo "Extension: card DRAW_THREE -- Example 2"
echo "================================================"
java -cp project-v2.jar:project-v1.jar uno.v2.Main \
     -p2 card DRAW_THREE input/deck2.txt input/script2.txt 2 > output/out2.txt

...
```

Any equivalent structure is acceptable, provided that the output of the script is organized, explicit, and easy for the instructors to follow (c.f. Section 4.1 - **Required ZIP structure**).

Important. Only the Phase 2 extensions and executions listed in `phase2-script.sh` will be considered for automatic inspection and evaluation. All required files must therefore be included in the submission and placed in the appropriate locations so that the script can be executed directly, without modification, and without changing file paths or required directories.

Required consistency. The Phase 2 execution script, the corresponding input files, the submitted report, and the actual executable behavior must be mutually consistent. In particular:

- every declared execution must be runnable;
- the corresponding input files must be included in the submission;
- the names used in the Phase 2 execution script must match the names accepted by the program;
- the report must briefly explain how each extension was implemented, how it integrates with the original design, and what each listed execution demonstrates (approximately 1–2 pages per extension).

Output format. Whenever possible, Phase 2 executions should preserve the same event-based output style used in Phase 1. This makes inspection easier and keeps the behavior of the project consistent across both phases.

4 Assessment and Grading

The project contributes **8 points out of 20** to the final course grade. To determine this contribution, the project will first be evaluated internally on a scale of 20 values and then converted into the corresponding project mark. The regular project grade is divided into two components:

Component	**Internal mark**	**Project mark**
UML	5 values	2.0 points
Implementation and behavior	15 values	6.0 points
Regular project total	**20 values**	**8.0 points**

Phase 1 may be implemented at different levels of completeness. This means that a student may choose to stop at a partial implementation and still obtain a lower, but valid, mark. The intended milestones are:

Classic UNO functionality implemented	**Internal mark**	**Project mark**
Number cards only	up to 17 values	up to 6.8 points
Number cards + `SKIP` and `REVERSE`	up to 18 values	up to 7.2 points
Add `DRAW_TWO`	up to 19 values	up to 7.6 points
Add `WILD` and `WILD_DRAW_FOUR`	up to 20 values	up to 8.0 points

These levels should be understood as internal completeness levels for the **Phase 1 implementation**. They indicate how complete the classic UNO engine is.

Phase 2 is evaluated as a **real bonus** on top of the **Project mark**. The following table summarizes the effect of the bonus:

Example situation	**Internal mark***	**Project mark**
Phase 1 complete, no Phase 2 extension	up to 20.0 values	up to 8.0 points
+ 1 meaningful Phase 2 extension	up to 20.5 values	up to 8.5 points
+ 2 meaningful Phase 2 extensions	up to 21.0 values	up to 9.0 points
Phase 1 up to `DRAW_TWO` only, no Phase 2 extension	up to 19.0 values	up to 7.6 points
+ 1 meaningful Phase 2 extension	up to 19.5 values	up to 8.1 points
+ 2 meaningful Phase 2 extensions	up to 20.0 values	up to 8.6 points
Phase 1 up to `SKIP`/`REVERSE` only, no Phase 2 extension	up to 18.0 values	up to 7.2 points
+ 1 meaningful Phase 2 extension	up to 18.5 values	up to 7.7 points
+ 2 meaningful Phase 2 extensions	up to 19.0 values	up to 8.2 points
Phase 1 up to Number cards only, no Phase 2 extension	up to 17.0 values	up to 6.8 points
+ 1 meaningful Phase 2 extension	up to 17.5 values	up to 7.3 points
+ 2 meaningful Phase 2 extensions	up to 18.0 values	up to 8.8 points

For instance, even if a group completes only Phase 1 up to `SKIP`/`REVERSE` (corresponding up to 18 internal values, i.e., up to 7.2 project points), it may still improve its final project mark by implementing meaningful Phase 2 extensions. In particular, if two strong and well-integrated extensions are delivered, the project mark may rise above the regular 8.0-point maximum of the project component.

This happens because, in the regular part of the project, the internal evaluation scale is converted linearly: each internal value up to 20 corresponds to 0.4 project points, yielding at most 8.0 project points. By contrast, the Phase 2 bonus is not subject to that same conversion factor. Instead, it is added directly as a bonus to the project mark.

The reason for this asymmetry is pedagogical: implementing meaningful extensions is considered a richer learning objective in this course, since it rewards not only correctness, but also abstraction, extensibility, and the ability to evolve a software design beyond its initial version.

4.1 Deadlines and Submission

The project must be submitted on Fenix **by June 5, 23:59 (anywhere on the planet)**. The submission must be organized as follows:

1. **Java — Phase 1**
   - The UML specification, including packages and classes, should be submitted in `.pdf` format:
     - only `.pdf` format is accepted,
     - The file must be named exactly `UML-v1.pdf`.
   - The executable JAR file named exactly `project-v1.jar`, including:
     - the corresponding Java source files (`.java`),
     - the compiled classes (`.class`).
   - Javadoc documentation, generated with the Javadoc tool, must be placed in a folder named `JDOC-v1`.
   - The `phase1-script.sh` file required for evaluation must be included inside a folder named `DEMO-v1`. Include also the `input`/`output` folders and respective files.
2. **Java — Phase 2** (if applicable)
   - The UML specification, including packages and classes, should be submitted in `.pdf` format:
     - only `.pdf` format is accepted,
     - The file must be named exactly `UML-v2.pdf`.
   - The executable JAR file must be named `project-v2.jar`, including:
     - the corresponding Java source files (`.java`),
     - the compiled classes (`.class`).
   - Javadoc documentation, generated with the Javadoc tool, must be placed in a folder named `JDOC-v2`.
   - The `phase2-script.sh` file required for evaluation must be included. It must contain five meaningful Phase 2 execution examples, together with the corresponding input files and the respective produced outputs, organized into separate `input` and `output` folders. All these materials must be placed inside a folder named `DEMO-v2`.
   - The report (max 1-2 pages per extension) in `.pdf` format named `report-v2.pdf`.

Required ZIP structure. All the required materials must be submitted through Fenix in a **single** `project-OOP-2526.zip` **file**. Only files with extension `.zip` will be accepted. Do not include your student numbers in the file name; Fenix will handle the identification automatically.

A typical submission structure for Phase 1 only may therefore look like:

```
project-OOP-2526.zip
  UML-v1.pdf
  project-v1.jar
  JDOC-v1/...
  DEMO-v1/phase1-script.sh
  DEMO-v1/input/...   <= the 5 examples from Fenix
  DEMO-v1/output/...  <= the corresponding output generated by phase1-script.sh
```

and if Phase 2 is also included:

```
project-OOP-2526.zip
  UML-v1.pdf
  UML-v2.pdf
  project-v1.jar
  project-v2.jar
  JDOC-v1/...
  JDOC-v2/...
  report-v2.pdf
  DEMO-v2/phase2-script.sh
  DEMO-v2/input/...   <= the 5 examples per extension, provided by the group
  DEMO-v2/output/...  <= the corresponding output generated by phase2-script.sh
```

Additional required actions. In addition to the ZIP file submission on Fenix, each group must also submit:

- **Self-evaluation form:**
  The self-evaluation form must be submitted on Fenix **by Monday, June 8, 14:00**.
- **Registration for the oral project discussion:**
  Registration must be completed on Teams **by Monday, June 8, 14:00**. Only one student per group should register the group. The selected slot may be changed later if other slots remain available, but any registration or later change must be agreed upon with the other group members in advance.

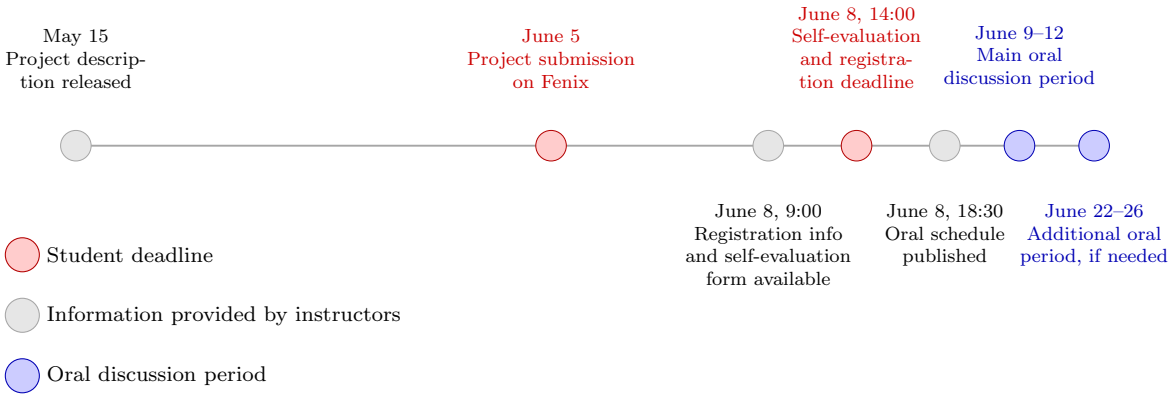
The relevant information for this process will be made available to students **by Monday, June 8, 9:00**. Please record these deadlines in your calendar if necessary.

4.2 Final Oral Discussion

The oral discussions are expected to take place on **June 9–12, 2026** and, only if necessary, also on **June 22–26, 2026**. The distribution of groups for the oral discussion will be made available on **June 8, 18:30**. All students should check this information before the discussions begin.

Important. All group members must be present during the oral discussion. The final project grade depends on this discussion, and it does **not** need to be the same for all members of the group. Moreover, regardless of the IDE used during development, all students are expected to be able to work with Java from the command line, including compiling, running, and discussing the submitted project in that context.

4.3 Timeline



4.4 Penalties

In addition to the grading scheme described above, the quality of the Java implementation, compliance with the project requirements, and the quality of the oral discussion are also important evaluation criteria.

4.4.1 Penalties Related to Automatic Assessment

The following penalties may therefore be applied when relevant:

- **Up to -1.0 values:** missing or unusable deliverables, such as a missing runnable artifact when one is required, missing source code, or a submission that cannot be compiled and inspected properly.
- **Up to -0.5 values:** submission format not respecting the requested structure or packaging instructions.
- **Up to -0.5 values:** hardcoded input files or hardcoded execution assumptions, when these violate the project requirements.
- **Up to -0.5 values:** output that does not follow the requested format, including stray prints, debugging information, or lines outside the specified structure.

These penalties are not intended to replace the regular grading criteria, but rather to reflect serious problems in the submission that hinder the automatic assessment of the proposed solution, namely its execution and the verification of its input/output behavior.

4.4.2 Individual Assessment in the Oral Discussion

The oral discussion includes an important **individual assessment component**, especially with respect to the **UML and Object-oriented design**. As a consequence, members of the same group do not necessarily receive the same mark. Individual marks may differ whenever the oral discussion reveals significantly different levels of understanding of the submitted solution.

The individual mark of each student will reflect that student's demonstrated understanding of the submitted solution, and not merely the set of implemented features delivered by the group. Therefore, if a significant discrepancy is observed between what was submitted and what a student is able to explain and justify during the oral discussion, the individual mark may differ accordingly. Students are expected to be familiar with the submitted work, to take responsibility for their contribution, and to act in accordance with the ethical standards of the course.

4.4.3 Penalty for Late Submission

Only Phase 1 submissions may be accepted with delay, subject to a late-submission penalty. **Phase 2 submissions for bonus evaluation, if provided, must be submitted by the official deadline and will not be accepted late.**

Projects submitted after the deadline incur a penalty. For each day of delay, a penalty of 2^{n-1} value will be deducted from the **Internal mark***, where n is the number of delayed days. Thus:

- 1 day late: penalty of $2^0 = 1$ value;
- 2 days late: penalty of $2^1 = 2$ values;
- 3 days late: penalty of $2^2 = 4$ values;
- and so on.

A day of delay is defined as a full 24-hour period counted from the official submission deadline.

These penalties apply to the **Internal mark*** (as in the table of the **Example situation**).